Learning the Graph for Relational Deep Learning

Milestone report

Harshal Kulkarni `hsk8171@stanford.edu`
Project 3: PR to `torch_geometric.contrib` + blog post

Code: github.com/HARSHALK2598/cs224w-structural-optimizer (experiments, plans, results)
github.com/HARSHALK2598/pytorch_geometric, branch `contrib/structural-optimizer` (the PyG contribution)

1 Problem and paper

Relational deep learning [2] turns a database into a graph mechanically (rows $\rightarrow$ nodes, foreign keys $\rightarrow$ edges). Cheng and Luo [1] argue this schema graph is often poor for message passing: irrelevant columns and tables add noise, and related rows sit many hops apart. Their structural optimizer learns the graph with the model: **A** filtering (Hard-Concrete [4] switches on columns and tables, plus a bottleneck), **B** injection (KNN, two-hop, behavioural and temporal edge templates), **C** a learned switch per injected relation inside aggregation, and **D** an L_0 penalty on open switches. On RelBench [3] it reports, e.g., `driver-top3` test ROC-AUC 81.5 (Base) $\rightarrow$ 84.7 (filtering) $\rightarrow$ 86.8 (full). We implement it as PyG `contrib` modules, written from the paper's equations, and test its claims.

2 Progress

Step	PyG classes	Status	Tests
A: column/table switches	`HardConcreteGate`, `RelationalFilter`	merged; full experiment	48
B: edge templates	`KNNEdges`, `TwoHopEdges`, `BehavioralEdges`, `TemporalContinuity`	implemented; PR in review	21
C+D: switch per relation	`GatedHeteroConv`	plan in review	–
A: bottleneck; wrapper	VIB, `StructuralOptimizer`	next	–

- **Leak-free injection.** With temporal sampling, a mini-batch is one subgraph per seed, each cut at its seed's time. An injected edge $u \rightarrow v$ is allowed only if u's subgraph is not newer than v's; two-hop shortcuts are composed per batch (built once on the database, a future race result would link a driver to their next team). Without this rule, about half the KNN edges in a test batch came from newer subgraphs.
- **Injected edges reach message passing:** each injected relation gets its own convolution, and a test checks that it receives a gradient. In the authors' code (read, not copied) they get no convolution, so injection cannot change predictions, and the template switches only feel the penalty.
- **Found along the way:** a PyTorch Frame bug gives a missing value's column a NaN gradient (fixed behind a flag); encoding each sampled row once per batch trains 2.5$\times$ faster; PyTorch Frame's column statistics copy values one by one (rel-event build 28.7 $\rightarrow$ 1.3 min).

3 Results: `rel-f1 driver-top3`

Protocol fixed in advance: tune Base on the paper's grid (learning rate $\times$ hidden size), tune each addition with the same budget, then compare on 5 fresh seeds (10–14); choose on validation, read test once. All runs: learning rate 10^{-2}, hidden 64, 30 epochs.

	Validation (ours)	Test (ours)	Δ val vs Base	Test (paper)
Base	0.785 ± 0.008	0.766 ± 0.040	–	0.815 ± 0.011
+ filtering, $\lambda = \lambda_s = 10^{-4}$	0.790 ± 0.014	0.772 ± 0.041	$+0.005$ ($p = 0.50$)	0.847 ± 0.006
+ KNN edges, $n = 10$, no switch	0.752 ± 0.013	0.784 ± 0.022	-0.033 ($p = 0.003$)	–

- **Filtering: no measurable gain** (the paper reports $+3.2$). λ does control sparsity (31, 25, 13 of 45 columns open for $\lambda = 10^{-4}, 10^{-3}, 10^{-2}$), but the switches close around epoch 14 while the best epoch is around 16, so the chosen models are barely filtered.
- **KNN alone lowers validation** by 0.033 ($p = 0.003$, Welch; test differs by $+0.019$, $p = 0.39$), although the paper finds KNN helps classification. This is what step C's per-relation switch is meant to handle. (Base ran on a GPU, KNN on a CPU: same code, so we compare means.)
- Our Base is about 5 points below the paper's on test (0.766 vs 0.815), with the paper's tuning grid.

4 Next steps

(1) Finish the implementation in small reviewed PRs: switches on injected relations with the λ_k penalty (C+D), the bottleneck, the `StructuralOptimizer` wrapper (every step a flag; with all flags off it must equal the plain GNN), docs and an example. **(2)** Evaluate on every RelBench dataset that fits in memory and its classification and regression tasks, with our own hyperparameter sweeps for every model, reporting next to the paper's numbers without adjustment. **(3)** The blog post.

References

[1] Y. Cheng and S. Luo. What makes a desired graph for relational deep learning? In *ICML*, 2026. arXiv:2606.08491.

[2] M. Fey, W. Hu, K. Huang, J. E. Lenssen, R. Ranjan, J. Robinson, R. Ying, J. You, and J. Leskovec. Position: Relational deep learning: Graph representation learning on relational databases. In *ICML*, 2024.

[3] J. Robinson, R. Ranjan, W. Hu, K. Huang, J. Han, A. Dobles, M. Fey, J. E. Lenssen, Y. Yuan, Z. Zhang, X. He, and J. Leskovec. RelBench: A benchmark for deep learning on relational databases. In *NeurIPS Datasets and Benchmarks*, 2024.

[4] C. Louizos, M. Welling, and D. P. Kingma. Learning sparse neural networks through L_0 regularization. In *ICLR*, 2018.